POLICY_MATCH_AI (GovSchemeAI)

Conversational Government Scheme Advisor Using RAG, Semantic Search & Large Language Models

Authors: Naveen Kumar K (252BDA25) • Dharnesh (252BDA27) • Prithvish William (252BDA18)
Program: MSc Big Data Analytics, St. Joseph's University • **Academic Year:** 2026

1. Executive Summary & Project Purpose

Policy_Match_AI is an AI-powered conversational platform engineered to solve the chronic problem of welfare scheme inaccessibility in India. While the Indian government operates thousands of central and state assistance programs, citizens struggle due to fragmented information, bureaucratic language, and complicated eligibility criteria.

The system allows citizens to speak or type in their everyday language (English, Hindi, Tamil, Kannada) describing their personal life situation. The platform dynamically extracts demographic attributes, performs dense semantic retrieval across **3,368+ schemes** using FAISS, and synthesizes step-by-step guidance via a grounded Large Language Model (Groq LLaMA 3.3 70B).

2. What We Have Built (System Overview)

Component	Technology / Model	Key Role in GovSchemeAI
Dataset	Cleaned CSV (3,368+ records)	Curated repository of Central & State government schemes, benefits, & criteria.
Vector Embeddings	SentenceTransformer (all-MiniLM-L6-v2)	Encodes scheme texts & user queries into 384-dimensional dense semantic vectors.
Semantic Search	FAISS (Facebook AI Similarity Search)	Sub-millisecond in-memory geometric distance search across vector coordinates.
Profile Extractor	Groq LLM + Regex Fallback	Extracts age, occupation, income, state, category from natural conversation.
Dialogue Manager	ConversationManager	Multi-turn context maintenance, follow-up detection, and session state.
LLM Generator	Groq (llama-3.3-70b-versatile)	Grounded plain-text generation in English, Hindi, Tamil, and Kannada.
Backend API	FastAPI (Python) + Uvicorn	REST API with CORS, /chat, and /new-session endpoints.
Mobile App	Flutter Cross-Platform	Chat UI, Voice STT/TTS, Interactive Video Avatar with camera, & Localization.

3. Quantitative Evaluation: How Did We Perform?

A formal Information Retrieval (IR) benchmark was implemented (rag_evaluator.py) to scientifically assess the quality and ranking precision of our retrieval engine:

Metric	Score	Benchmark Standard	Interpretation in GovSchemeAI

NDCG@5	**0.8324**	> 0.80 = Excellent	Top-5 recommended schemes are over 83% optimally ranked by relevance.
MRR	**0.8100**	> 0.70 = High Precision	The first strictly relevant scheme appears on average at position 1.2.
Precision@5	**0.8400**	> 0.75 = Strong	84% of all schemes returned in the top-5 list are directly relevant.
Recall	**0.7600**	> 0.70 = Good	Captures 76% of all eligible schemes in the knowledge base.
Hit Rate	**100.0%**	100% = No Zero-Hits	Every single query successfully located at least one relevant scheme.

4. Technical Deep-Dive: Architecture Q&A; & Key Discussions

Q1: Is FAISS an analysis tool or a database? What does it actually store?

FAISS (Facebook AI Similarity Search) is an **in-memory vector indexing library**. It does *not* store pre-calculated queries or distances to queries. Instead, it stores the raw **384-dimensional mathematical coordinates (embeddings)** of all 3,368 government schemes in RAM:

- **At Startup:** The index is read from scheme_index.faiss on disk into RAM once. During live searches, it operates entirely in RAM in less than 2 milliseconds.
- **During Query:** When a user asks a question, SentenceTransformer converts the query text into a 384-dimensional vector. FAISS computes the Euclidean L2 distance between the query vector and all stored scheme vectors to find the nearest points.

Q2: If profile keywords are extracted, why do we need Semantic Search? Aren't we doing keyword search?

This addresses the core difference between **Demographics (Who)** and **Conceptual Intent (What)**:

1. The Vocabulary Mismatch Problem: Citizens use everyday conversational phrasing, whereas government policies use formal legal terminology. For instance, if a farmer says *'My crops got ruined in the flood'*, keyword search searches for words like *'ruined'* and *'flood'*. The official scheme is named *'Pradhan Mantri Fasal Bima Yojana: Agricultural Insurance against adverse weather calamities'*. Keyword search returns **0 matches**. Semantic search recognizes that 'ruined crops' and 'adverse weather insurance' occupy adjacent coordinates in vector space.

2. Demographic Filtering Alone is Insufficient: Over 400 schemes mention the keyword 'farmer' and 300 mention 'Karnataka'. Profile keywords alone cannot rank which of those 400 schemes addresses the citizen's specific hardship.

Q3: How is semantic meaning actually fetched from the unified query?

In rag_engine.py, the query text is processed by SentenceTransformer('all-MiniLM-L6-v2'). The transformer passes the phrase through **6 multi-head self-attention neural network layers**. These layers evaluate syntactic and semantic dependencies between words, compressing the entire conceptual meaning into a 384-dimensional embedding vector. FAISS then computes:

$$\text{Distance } d = \sqrt{\sum_{i=1}^{384} (Q_i - S_i)^2} \quad \implies \quad \text{Similarity} = \frac{1}{1 + d}$$

A smaller distance signifies that the two concepts reside in the same conceptual neighborhood.

Q4: Code Insight — Are we only sending profile keywords as the query?

Yes, in the current implementation of search_by_profile(): The code maps demographics into descriptive keyword phrases (e.g., *'farmer agriculture subsidy financial assistance Karnataka'*). While this finds general agricultural schemes, it can drop specific user intent (e.g., if the user asked specifically for a **'solar pump'**).

Recommended Upgrade: Modify search_by_profile() to combine the original user prompt with the profile keywords:
unified_query = f"{user_message} {profile_keywords}"
Result: Captures the specific intent ('solar pump') alongside the eligibility context ('farmer in Karnataka'), leading to exact scheme retrieval (e.g., PM-KUSUM Solar Scheme).

5. Current Project Status & Actionable Polish Items

Item / File	Current State	Action Required
Hardcoded Base Path src/app.py:30	Points to local folder of previous developer.	Use dynamic path: os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
API Base URL api_services.dart:8	Configured to an expired localtunnel URL.	Point to active server IP or http://10.0.2.2:8000 for emulator testing.
History Screen history_screen.dart	Empty file (0 bytes).	Implement simple ListView to display cached previous user conversations.
RAG Query Enrichment src/app.py & rag_engine.py	Uses profile keywords only.	Pass user_message + profile_keywords for high-specificity retrieval.

Report Generated: October 2026 • GovSchemeAI Research & Engineering Team